UTEC Universidad Tecnológica | ANEP | UTU DIRECCIÓN GENERAL DE EDUCACIÓN TÉCNICO PROFESIONAL

Evidencias 2 – Tacómetro Digital

Nombres y Apellidos	Sergio Romero / Anthony Bonino / Alberto Gomez
C.I.	4.444.574-9 / 4.862.676-7 / 4.862.694-9
ITR / Sede	Suroeste – Fray Bentos
Carrera	Ingeniería en Mecatrónica
Unidad Curricular	Proyecto Integrador de Competencias II
Docentes	Leonardo Eguia / Marcelo Díaz / Andrés Hippa / Rodrigo Sosa
Fecha de presentación	24/ 10 / 2026

Evidencias 2

En las Evidencias 1 se presentaron los primeros avances del proyecto **Tacómetro Digital**: el primer montaje del NodeMCU ESP8266 con el módulo infrarrojo FC-51, una primera versión del firmware, el diseño CAD de la carcasa (146 mm $\times$ 96 mm $\times$ 43 mm) y el esquemático preliminar en Proteus. En aquella instancia la simulación no pudo realizarse por problemas con el software.

El presente documento reúne las evidencias de la segunda etapa y continúa ese trabajo. Desde entonces se realizó la simulación en Wokwi, se completó el firmware, se incorporaron al prototipo la pantalla LCD y los elementos de interfaz, se definió la alimentación con una pila 18650 y se imprimió en 3D la carcasa. El principio de medición se mantiene: el FC-51 detecta el paso de una marca reflectante y el ESP8266 calcula las revoluciones por minuto (RPM) a partir del tiempo entre pasos sucesivos.

Para esta instancia se solicita un archivo comprimido `.rar` con el avance del diseño CAD y un documento `.pdf` con los siguientes puntos, que se desarrollan en el mismo orden en las secciones de este informe:

1. Estado del prototipo/simulación (avance actual y estética).
2. Datos de pruebas preliminares y análisis de los resultados obtenidos.
3. Diseño CAD (avances o correcciones).
4. Esquemático eléctrico actualizado.
5. Código utilizado en los ensayos con los comentarios más importantes.
6. Enlace a un video de las pruebas del prototipo/simulación y toma de datos.
7. Enlace a la simulación activa Wokwi.

El código fuente completo puede consultarse en el repositorio de GitHub del proyecto, cuyo enlace se encuentra en la Sección 8.

1. Estado del prototipo y de la simulación

La Tabla 1 compara el estado de cada parte con el presentado en las Evidencias 1.

Parte	Evidencias 1	Evidencias 2
Simulación	No realizada.	Realizada en Wokwi y funcionando.
Firmware	Versión inicial, sin pantalla.	Completo: estabilidad, retención del resultado, calibración, LCD y alarmas.
Prototipo	ESP8266 y FC-51 en protoboard.	Con LCD 16×2, LEDs, buzzer y tres pulsadores.
Carcasa	Diseño CAD.	Impresa en 3D, con los componentes montados y en funcionamiento.
Alimentación	No definida.	Una pila recargable de litio 18650 de 3,7 V, con cargador externo.

Tabla 1: Avance respecto de las Evidencias 1.

1.1 Simulación en Wokwi

Como Wokwi no incluye la placa ESP8266, la simulación se realizó con una placa **ESP32 DevKit V1**, con el programa adaptado a sus pines. El sensor FC-51 tampoco existe en el simulador y se reemplazó por un **generador de pulsos** (*clock generator*), que equivale al paso periódico de la marca reflectante. La Tabla 2 compara las conexiones de la simulación con las del circuito real.

Elemento	Simulación (ESP32)	Circuito real (ESP8266)
Señal del sensor	D27 (generador de pulsos)	D5 / GPIO14 (FC-51)
Pulsador INICIO	D32	D6 / GPIO12
Pulsador CAL	D33	D7 / GPIO13
Pulsador REINICIO	D26	RST
LCD (SDA / SCL)	D21 / D22	D2 / D1
LED rojo / verde / azul	D18 / D19 / D23, con 330 Ω	D3 / D4 / D0, con 330 Ω
Buzzer	D25, conectado directamente	D8 / GPIO15, mediante transistor

Tabla 2: Conexiones de la simulación frente al circuito real.

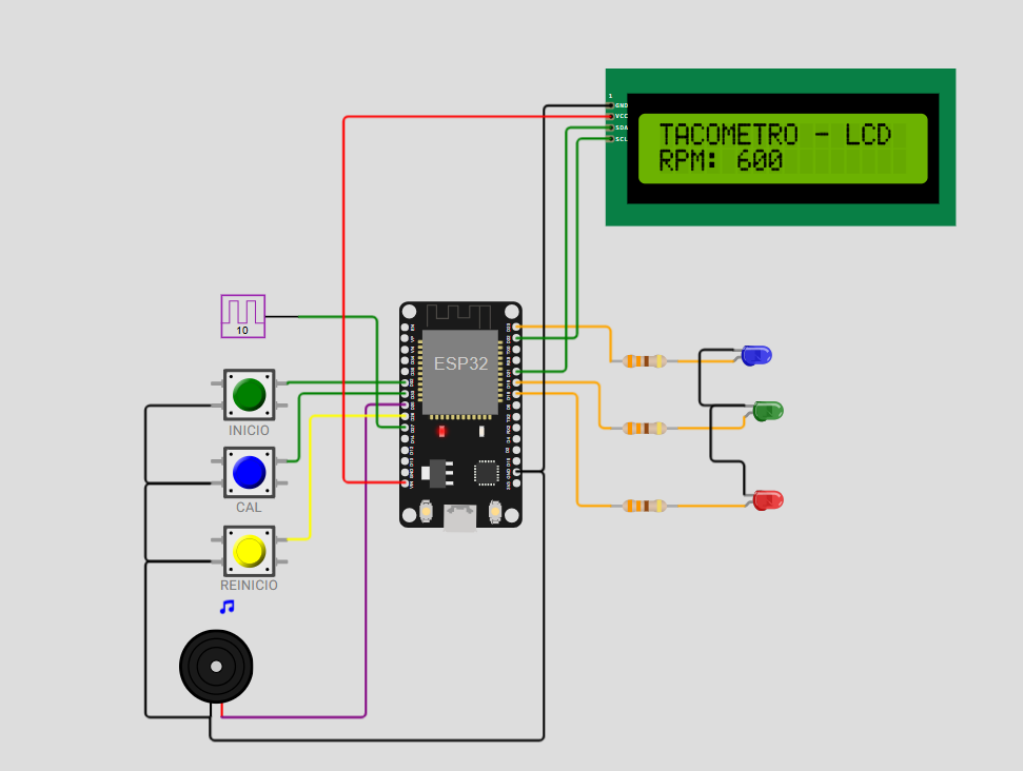


Figura 1: Simulación en Wokwi: con el generador a 10 Hz la pantalla indica 600 RPM.

La Figura 1 muestra la pantalla LCD 16×2 con adaptador I2C, los pulsadores INICIO, CAL y REINICIO, los tres LEDs y el buzzer. Con el generador en 10 Hz y una marca por vuelta se obtienen $10 \times 60 = 600$ RPM, el valor que indica la pantalla. La simulación verifica la lógica del programa, pero no el comportamiento real del sensor: la señal es ideal, sin ruido ni rebotes, y los pines no son los del ESP8266 (en la simulación los tres LEDs son activos en alto y el buzzer no usa transistor). Por eso la medición se valida con el prototipo real.

1.2 Prototipo en protoboard

Respecto del primer montaje de las Evidencias 1, el prototipo (Figura 2) incorpora la pantalla LCD con adaptador I2C (PCF8574), los LEDs rojo, verde y azul, el buzzer con su transistor y los pulsadores. El FC-51 se montó sobre una segunda protoboard para ubicarlo libremente frente a la marca. En la captura, la pantalla muestra *SIN CALIBRAR*: el equipo completó una medición estable, pero aún no tiene una calibración guardada.

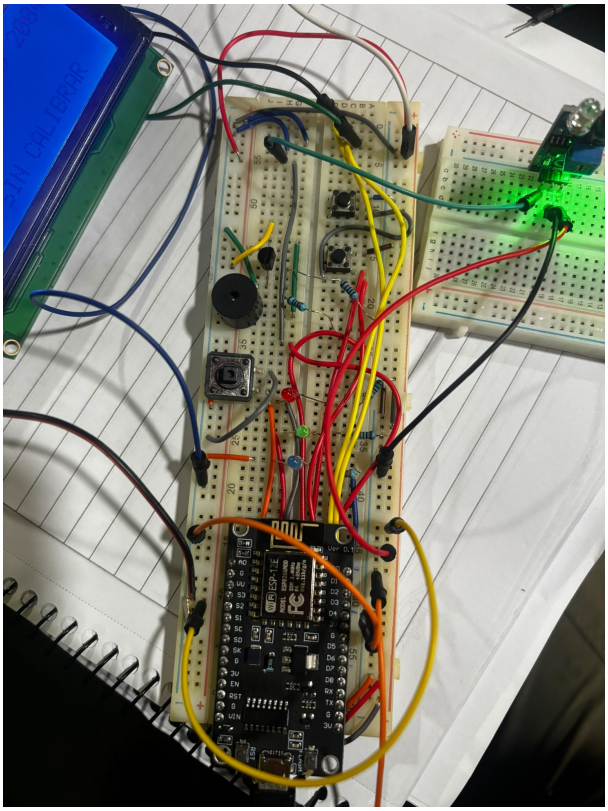


Figura 2: Prototipo del tacómetro digital en protoboard.

1.3 Estética: carcasa impresa

La carcasa diseñada en CAD (Sección 3) se imprimió en 3D con filamento blanco. La Figura 3 muestra una pieza durante la impresión, con la ventana rectangular prevista para la fuente de alimentación, que se imprime con soportes, y los postes de fijación. La Figura 4 muestra el cuerpo terminado, con los alojamientos para el módulo infrarrojo y la alimentación. La pantalla LCD y el buzzer quedan en la tapa frontal y el sensor FC-51 sobresale por el extremo superior de la carcasa, para dirigirlo hacia el eje a medir. Los componentes ya se montaron en la carcasa y el equipo funciona en su interior.

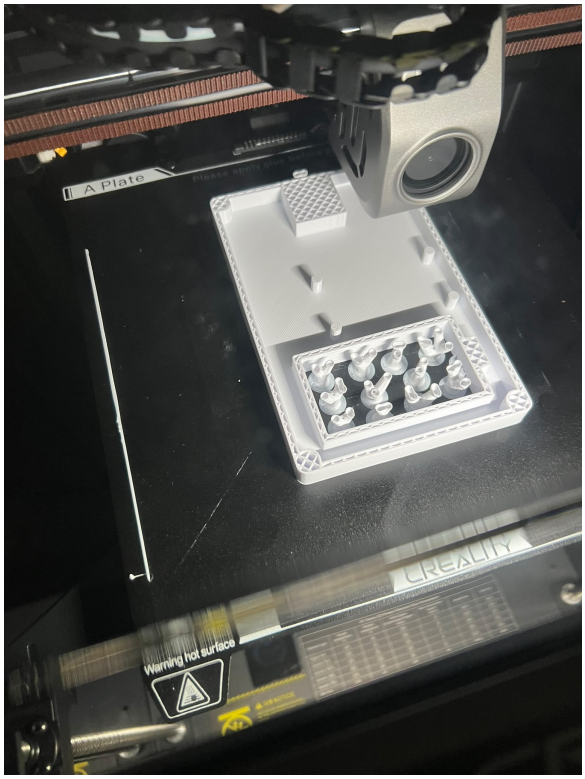


Figura 3: Pieza durante la impresión 3D.

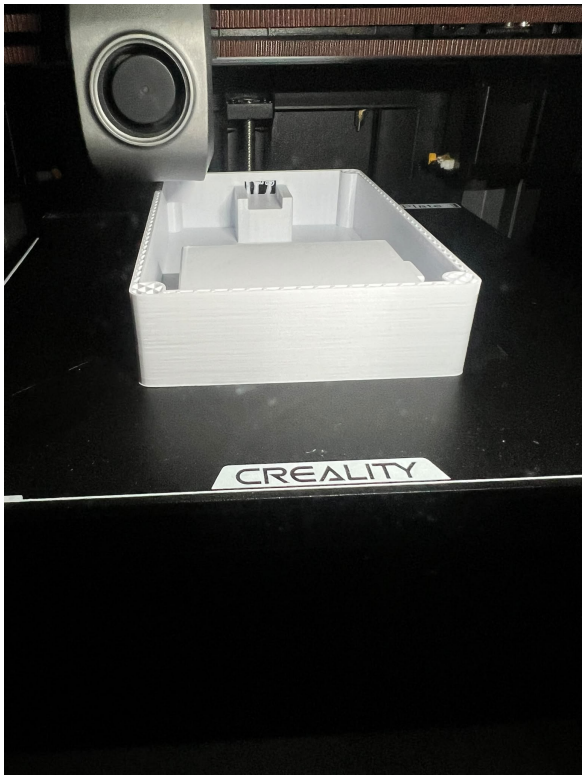


Figura 4: Carcasa terminada.

1.4 Alimentación

Para que el equipo sea portátil se utilizará **una pila recargable de litio 18650** de 3,7 V, del kit de la marca Alextremo que incluye dos pilas y su cargador (Tabla 3). Se usa una sola pila para alimentar todo el sistema; la segunda queda como repuesto, de modo que una puede cargarse mientras la otra está en uso.

Característica	Valor
Formato y química	18650, litio (Li-ion)
Tensión nominal	3,7 V (4,2 V con carga completa)
Capacidad indicada por el vendedor	7 500 mAh
Cargador	Externo, alimentado a 110/220 V, con dos puertos e indicador de carga; sin carga por USB
Contenido del kit	Cargador y dos pilas 18650

Tabla 3: Características del kit de alimentación seleccionado.

La pila se coloca en un portapilas 18650 dentro del alojamiento previsto en la carcasa y se conecta a la entrada VIN del NodeMCU. El regulador de la placa obtiene a partir de ella

los 3,3 V con los que se alimentan el ESP8266, el FC-51 y la pantalla LCD, de modo que todo el circuito trabaja con el mismo nivel lógico y no se necesitan módulos adicionales. La pila no se conecta directamente a los pines de 3,3 V, porque cargada llega a 4,2 V y el microcontrolador admite como máximo 3,6 V. Como el cargador es externo, para recargarla se retira la pila del equipo; así el circuito no necesita un módulo de carga propio.

El consumo estimado del circuito es de unos 100 mA (NodeMCU 30–40 mA, FC-51 20 mA, LCD 15–25 mA, LED 5 mA y buzzer solo durante los pitidos), y el regulador lineal consume de la pila la misma corriente que entrega. La capacidad de 7 500 mAh indicada por el vendedor es mayor que la de las pilas 18650 habituales, que suele estar entre 2 000 y 3 500 mAh, por lo que para el cálculo se adopta un valor conservador de 2 000 mAh:

$$T = \frac{2\,000\ \text{mAh}}{100\ \text{mA}} \approx 20\ \text{h}.$$

En la práctica se esperan más de 12 horas de uso continuo, suficientes para un instrumento que se enciende solo para medir. A medida que la pila se descarga, la tensión de 3,3 V también baja, por lo que conviene recargarla antes de que llegue a unos 3,5 V. La autonomía real se verificará midiendo el consumo del prototipo con un multímetro.

Además, se prevé agregar un **diodo zener** en la entrada de alimentación, en paralelo con la pila, para limitar la tensión que llega a la placa. Con la pila completamente cargada (4,2 V) o ante picos al conectarla, el zener conduce y evita que la tensión de entrada supere su valor nominal. Su tensión y potencia se definirán al medir el consumo real del prototipo, ya que el zener debe soportar la corriente que deriva sin afectar la autonomía.

2. Datos de pruebas preliminares y análisis

2.1 Pruebas en simulación

Se varió la frecuencia del generador y se comparó la lectura con el valor teórico RPM $= 60 \cdot f/\text{PPR}$, con PPR = 1 (Tabla 4).

Frecuencia (Hz)	RPM teóricas	RPM en pantalla	Error (%)
1	60	60	0,0
5	300	300	0,0
10	600	600	0,0
50	3 000	3 000	0,0
100	6 000	6 000	0,0
200	12 000	12 000	0,0

Tabla 4: Resultados de la simulación.

En todo el rango de 60 a 12 000 RPM la lectura coincide con el valor teórico, lo que confirma que la fórmula de cálculo es correcta. Este resultado es esperable, porque la señal simulada es ideal y el temporizador tiene una resolución de 1 µs.

2.2 Análisis

Resolución. Con una resolución de 1 µs, el error en el peor caso (12 000 RPM, período de 5 000 µs) es de 0,02 %, y al sumar todos los períodos de la ventana no se acumula. Los errores reales provendrán del sensor o de la marca, no del cálculo.

Ancho de la marca. El filtro exige 2 ms en cada nivel de la señal. Si la marca ocupa una fracción w de la circunferencia, la velocidad máxima medible es

$$\mathrm{RPM}_{\mathrm{máx}} = \frac{30\,000 \cdot \mathrm{mín}(w,\, 1-w)}{\mathrm{PPR}}.$$

Una cinta angosta ($w = 0{,}05$) solo permite medir hasta 1 500 RPM; con $w = 0{,}25$ se llega a 7 500 RPM, y para cubrir las 12 000 RPM la marca debe ocupar entre el 40 y el 60 % de la circunferencia. Este es el principal criterio a tener en cuenta al preparar los ensayos.

Velocidad mínima y tiempo de respuesta. A 60 RPM hay un solo período por ventana, por lo que la estabilización es más lenta; por debajo de unas 20 RPM los pasos de la marca se separan más de 3 s y el equipo indica *SIN SENAL*. Para velocidades bajas conviene usar varias marcas por vuelta (comando `PPR`). En condiciones normales el resultado se obtiene entre 3 y 4 s después de detectar la marca.

3. Diseño CAD

Se mantiene el diseño presentado en las Evidencias 1, con dimensiones de referencia de 146 mm × 96 mm × 43 mm. La Figura 5 muestra la distribución de los componentes. En la base (vista izquierda) se ubican el alojamiento para la pila 18650, el ESP8266 (módulo ESP-12E/ESP-12F) en la zona central y el módulo FC-51, basado en el comparador LM393, fijado en un soporte junto al borde para que el emisor y el receptor queden fuera de la carcasa. En la tapa (vista derecha) se encuentran la ventana de la pantalla LCD 16×2 y el orificio del buzzer.

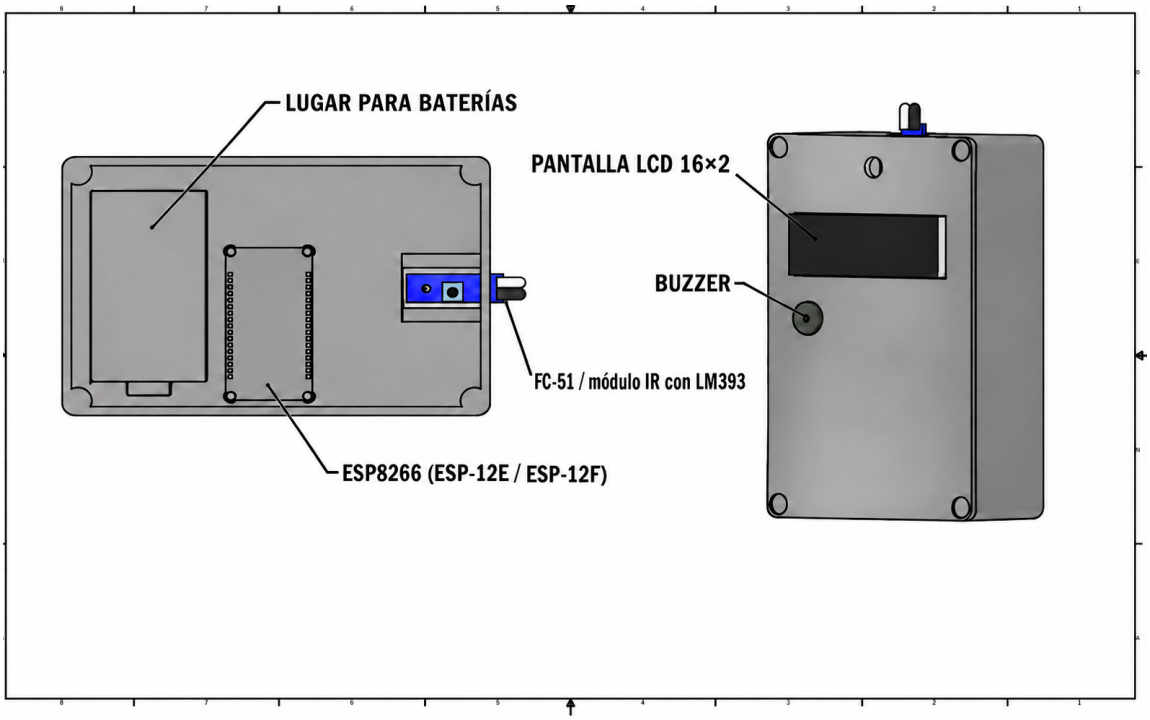


Figura 5: Distribución de los componentes en la base (izquierda) y en la tapa (derecha) de la carcasa.

Al incorporar los nuevos componentes del prototipo, que no figuran en el modelo original, se revisan los siguientes puntos del diseño:

- Orificios para los tres pulsadores (INICIO, CAL y REINICIO) y los tres LEDs.
- Espacio para el transistor del buzzer y para el portapilas 18650 (la pila mide unos 18 mm de diámetro y 65 mm de largo), con acceso para retirarla y cargarla.
- Ventana frontal para el emisor y el receptor del FC-51 y acceso a su potenciómetro.

4. Esquemático eléctrico actualizado

El esquemático de la Figura 6 reemplaza al preliminar de las Evidencias 1. Corresponde al circuito real con el ESP8266 (no al de la simulación) y coincide con el mapa de pines del firmware (Código 1).

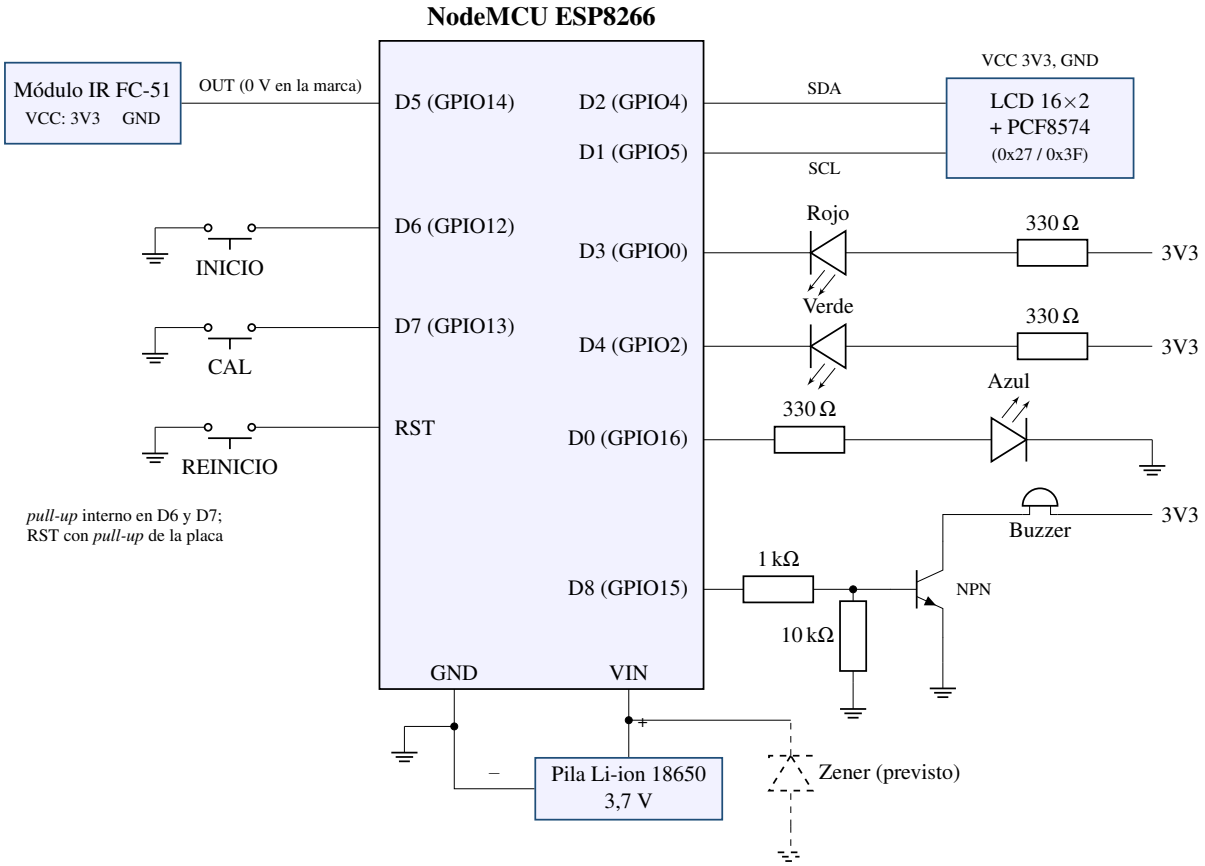


Figura 6: Esquemático eléctrico actualizado del tacómetro digital con NodeMCU ESP8266.

Los pines respetan las condiciones de arranque del ESP8266, que requiere GPIO0 y GPIO2 en alto y GPIO15 en bajo al encender: por eso los LEDs rojo y verde se conectan desde 3V3 (activos en bajo) y la resistencia de 10 kΩ mantiene en bajo la base del transistor del buzzer. La pila 18650 alimenta la placa por VIN, con un diodo zener previsto en paralelo para limitar la tensión de entrada (en línea discontinua), y el FC-51 y el LCD se alimentan desde 3V3, por lo que la salida del sensor y el bus I2C trabajan al mismo nivel que el ESP8266.

5. Código utilizado en los ensayos

El firmware está escrito en C para el NodeMCU 1.0 (ESP8266EX), con el entorno ESP8266_RTOS_SDK v3.4, y se divide en `configuracion.h` (pines), `captura.h` (períodos, RPM y regularidad) y `main.c` (interrupción, estabilidad, calibración, pantalla, indicadores y bucle principal). La Tabla 5 resume su evolución respecto de la versión de las Evidencias 1.

Aspecto	Evidencias 1	Evidencias 2
Interrupción	Flanco negativo, descarte de pulsos menores de 200 µs.	Ambos flancos; exige 2 ms en cada nivel.
Cálculo de RPM	Promedio de un arreglo circular.	Suma de períodos de una ventana de 1 s.
Estabilidad	Cuatro muestras, 5 % y 1,5 s.	Tres ventanas coherentes (5 %), regularidad y 1,5 s; retiene el primer resultado estable.
Calibración	Factor fijo.	Con referencia real, guardada en memoria NVS.
Interfaz	LEDs, pulsadores y buzzer.	Además, LCD por I2C, comandos por UART y alarma sonora.

Tabla 5: Evolución del firmware respecto de las Evidencias 1.

5.1 Mapa de pines

Código 1: Definición de pines (`configuracion.h`)

```
#define SENSOR 14       /* D5, salida digital del sensor, maximo 3.3 V */
#define INICIO 12       /* D6, pulsador hacia GND */
#define CAL 13          /* D7, pulsador hacia GND */
#define LCD_SDA 4       /* D2 */
#define LCD_SCL 5       /* D1 */
#define ROJO 0          /* D3, LED activo en bajo */
#define VERDE 2         /* D4, LED activo en bajo */
#define AZUL 16         /* D0, LED activo en alto */
#define BUZZER 15       /* D8, transistor, activo en alto */
/* Tercer pulsador: RST a GND. No ocupa un GPIO. */
```

5.2 Detección de la marca

Código 2: Interrupción del sensor con filtro de 2 ms (`main.c`)

```
static void pulso(void *arg) {
    int nivel=gpio_get_level(SENSOR);
```

```
    int64_t ahora=esp_timer_get_time();
    if(nivel==sensor_nivel) return;
    int64_t duracion=ahora-sensor_cambio;
    if(!nivel) {
        /* Armar solo si previamente hubo salida estable de la marca. */
        sensor_armado=marcha && duracion>=SENSOR_ESTABLE_US;
    } else {
        /* Confirmar la marca completa; medir entre sus flancos de entrada. */
        if(marcha && sensor_armado && duracion>=SENSOR_ESTABLE_US)
            captura_evento(&captura,sensor_cambio,ppr);
        sensor_armado=false;
    }
    sensor_nivel=nivel; sensor_cambio=ahora;
}
```

El sensor entrega 0 V sobre la marca y unos 3 V fuera de ella. La interrupción se ejecuta en ambos flancos: al entrar en la marca se *arma* solo si la señal estuvo al menos 2 ms en alto, y al salir se confirma el evento solo si estuvo al menos 2 ms en bajo. Se registra el instante de entrada, de modo que el período se mide entre entradas consecutivas y cada marca genera un único evento; los rebotes de menos de 2 ms se descartan. Luego, la función `captura_evento()` acumula la cantidad de períodos, su suma, el mínimo, el máximo y los descartados, y reinicia las estadísticas si pasan 3 s sin marcas.

5.3 Cálculo de RPM, estabilidad y retención

Código 3: Cálculo de RPM y regularidad (`captura.h`)

```
r.cruda=(float)(60000000.0*(double)c->periodos/((double)c->suma_us*ppr));
r.rpm=r.cruda*factor;
r.rango=r.rpm>=60.0f && r.rpm<=12000.0f;
/* No declarar estable una entrada con muchos pulsos descartados. */
r.regular=(c->maximo_us-c->minimo_us)<=promedio*0.10 &&
          (uint64_t)c->descartados*4<=c->periodos;
```

Cada segundo se calculan las RPM con la suma de los períodos de la ventana:

$$\mathrm{RPM} = \frac{60\,000\,000 \cdot N_{\mathrm{per}}}{\Sigma t_{\mu s} \cdot \mathrm{PPR}} \cdot f_{\mathrm{cal}},$$

donde N_{per} es la cantidad de períodos, $\Sigma t_{\mu s}$ su suma en microsegundos, PPR las marcas por vuelta y f_{cal} el factor de calibración. Por ejemplo, a 10 Hz hay 10 períodos de 100 000 µs y resultan 600 RPM. Sumar los tiempos, en lugar de contar pulsos por segundo, mejora la re-

solución a baja velocidad. La ventana es *regular* si la dispersión de los períodos no supera el 10 % del promedio y los descartados no superan el 25 %.

Código 4: Criterio de estabilidad y retención del resultado (`main.c`)

```
estable=candidato && ventanas_estables>=3 && ahora-desde_estable
   >=1500000;
/* Calibrar necesita captura continua para poder recibir REF.
   En medicion normal, retener el primer resultado estable. */
if(estable && !calibrando) {
    ultima_rpm=rpm; lectura_guardada=true; medida_finalizada=true;
    marcha=false; pitido_fases=0;
}
```

Una lectura es estable con tres ventanas consecutivas en rango, regulares y con una variación de hasta 5 %, durante al menos 1,5 s. En medición normal, el primer resultado estable queda fijo en pantalla y la medición se detiene hasta pulsar INICIO; en calibración la captura continúa. El resultado retenido se guarda en RAM, por lo que se pierde al apagar o reiniciar el equipo; en cambio, el factor de calibración y el PPR se conservan en la memoria no volátil.

5.4 Calibración y memoria no volátil

Código 5: Calibración mediante el comando REF (`main.c`)

```
} else if(!strncmp(s,"REF ",4)) {
    char *fin;float ref=strtof(s+4,&fin);float f=cruda>0?ref/cruda:0;
    if(*fin || !calibrando || !estable || !isfinite(ref) ||
        ref<60 || ref>12000 || f<0.5f || f>2.0f) {
        printf("REF rechazada: activar CAL, estabilizar y usar
           referencia real\n");
    } else {
        factor=f;calibrada=respaldo=true;factor_ref=f;ppr_ref=ppr;
        calibrando=false;guardar();
    }
}
```

Con el modo calibración activo y una lectura estable de una velocidad conocida, se envía por puerto serie `REF <rpm>`. El factor $f_{\text{cal}} = \text{RPM}_{\text{ref}}/\text{RPM}_{\text{cruda}}$ se acepta solo entre 0,5 y 2. La función `guardar()` lo almacena junto con el PPR en la memoria flash (NVS), deshabilitando la interrupción durante la escritura y restaurando la configuración anterior si esta falla; así la calibración se conserva tras un reinicio. Una copia de respaldo puede recuperarse con `RESTAURAR` o manteniendo CAL durante 3 s.

5.5 Interfaz con el usuario

La pantalla LCD 16×2 se maneja en modo de 4 bits a través del PCF8574, cuya dirección (0x27, 0x3F u otra) se detecta automáticamente. Cada 250 ms la primera fila muestra las RPM y la segunda el estado: *DETENIDO*, *ESTABILIZANDO*, *ESTABLE - INICIO*, *SIN CALIBRAR*, *CALIBRACION*, *SIN SENAL*, *FUERA DE RANGO* o *SENAL IRREGULAR* (sin tildes, porque el controlador no las incluye). Si el LCD falla, se deshabilita sin detener la medición. El LED verde indica medición calibrada sin errores, el azul el modo calibración y el rojo una alarma (sin señal, fuera de rango, irregular o sin calibrar). El buzzer, excitado con PWM de 2 kHz, da un pitido al iniciar, dos al entrar en calibración y pitidos repetidos durante la alarma, sin bloquear la medición. Los pulsadores tienen antirrebote de 40 ms y por el puerto serie (115 200 baudios) se aceptan los comandos de la Tabla 6. Además, cada 250 ms se imprime un diagnóstico usado en la toma de datos.

Comando	Función
`INICIO` / `PARAR`	Inicia una nueva medición o la detiene conservando la última lectura.
`CAL`	Activa o desactiva el modo calibración.
`REF <rpm>`	Calibra con una velocidad real conocida (requiere calibración activa y lectura estable).
`PPR <1..20>`	Define las marcas por vuelta (con el equipo detenido; borra la calibración).
`RESTAURAR`	Recupera la calibración de respaldo.

Tabla 6: Comandos por puerto serie.

5.6 Diagramas de funcionamiento

La Figura 7 muestra la arquitectura del sistema, la Figura 8 el flujo del programa principal y la Figura 9 los modos de funcionamiento.

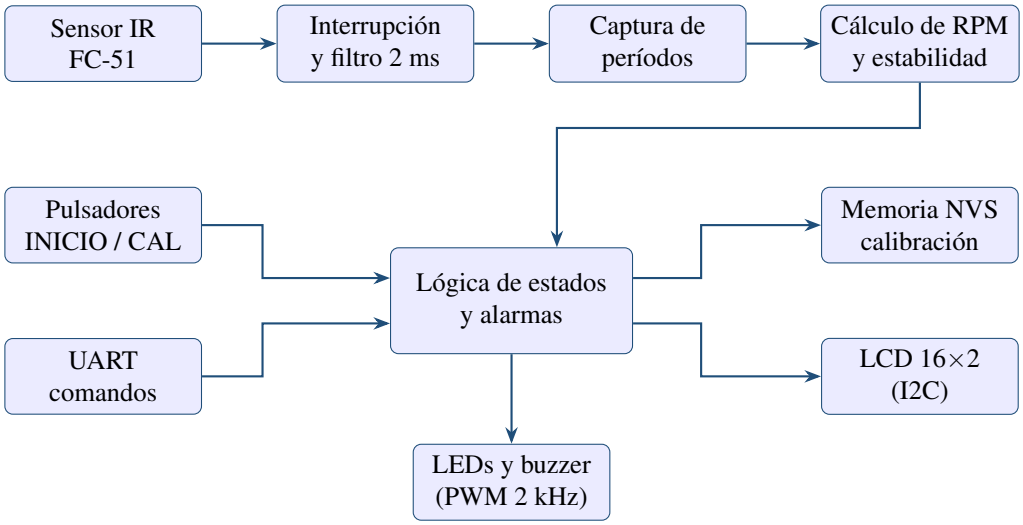


Figura 7: Diagrama de bloques del tacómetro.

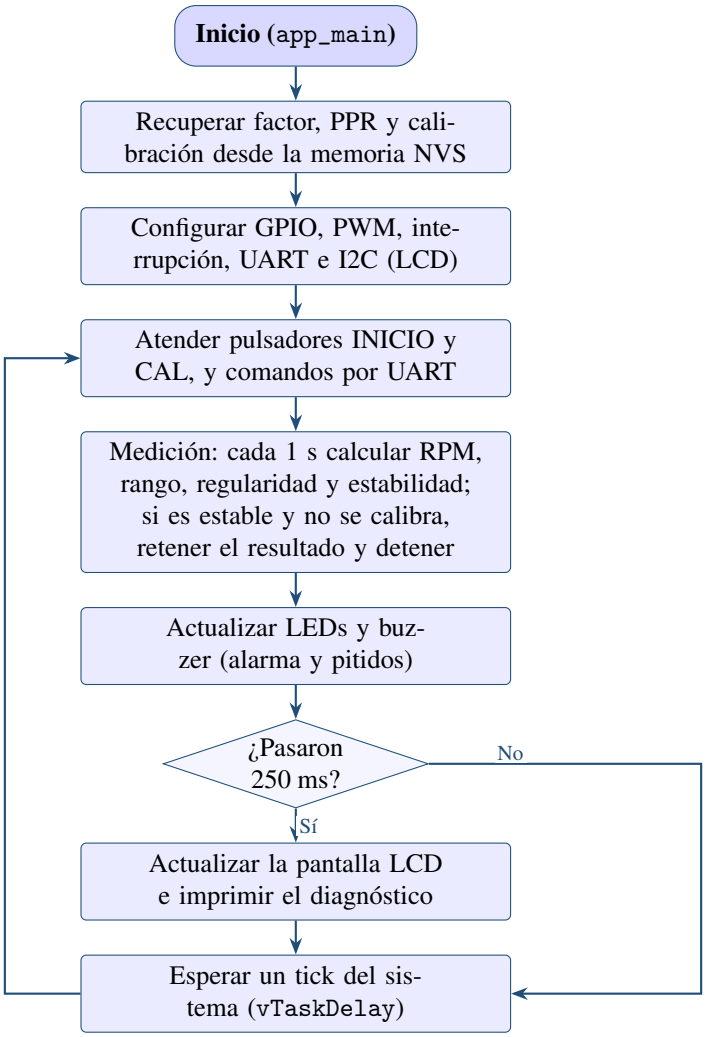


Figura 8: Diagrama de flujo del programa principal.

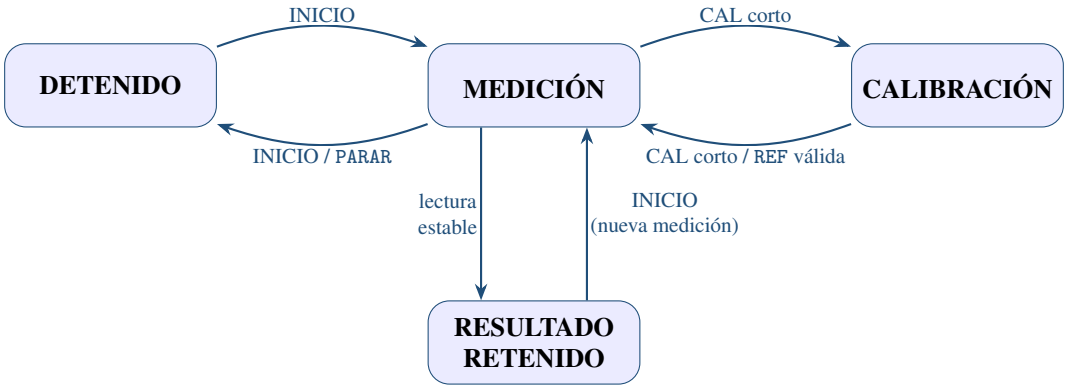


Figura 9: Modos de funcionamiento del equipo.

En *MEDICIÓN* el equipo pasa a *RESULTADO RETENIDO* al obtener la primera lectura estable; en *CALIBRACIÓN* espera el comando `REF`. La alarma no es un modo aparte: se superpone a la medición cuando, pasados 3 s desde el inicio, hay falta de señal, lectura fuera de rango, pulsos irregulares o el equipo no está calibrado.

6. Video de las pruebas en simulación

El siguiente video muestra el circuito simulado en Wokwi y su funcionamiento: se inicia la medición, se modifica la frecuencia del generador de pulsos y se observa la lectura de RPM en la pantalla LCD. Las pruebas con el prototipo real se documentan con las fotografías de la Sección 1.

Video explicativo de la simulación en Wokwi

Abrir video en Google Drive

7. Enlace a la simulación activa

La simulación se encuentra publicada en Wokwi, un simulador en línea similar a TinkerCAD que permite trabajar con placas ESP32. Para probarla se inicia el simulador, se pulsa INICIO y se modifica la frecuencia del generador de pulsos para observar el cambio en la lectura (RPM = 60 $\times$ frecuencia en Hz).

Simulación del tacómetro digital en Wokwi

Abrir simulación en Wokwi

8. Repositorio del proyecto

El código fuente completo del firmware (`configuracion.h`, `captura.h` y `main.c`), junto con el programa utilizado en la simulación, se encuentra disponible en el siguiente repositorio de GitHub.

Repositorio del Tacómetro Digital en GitHub

Abrir repositorio en GitHub